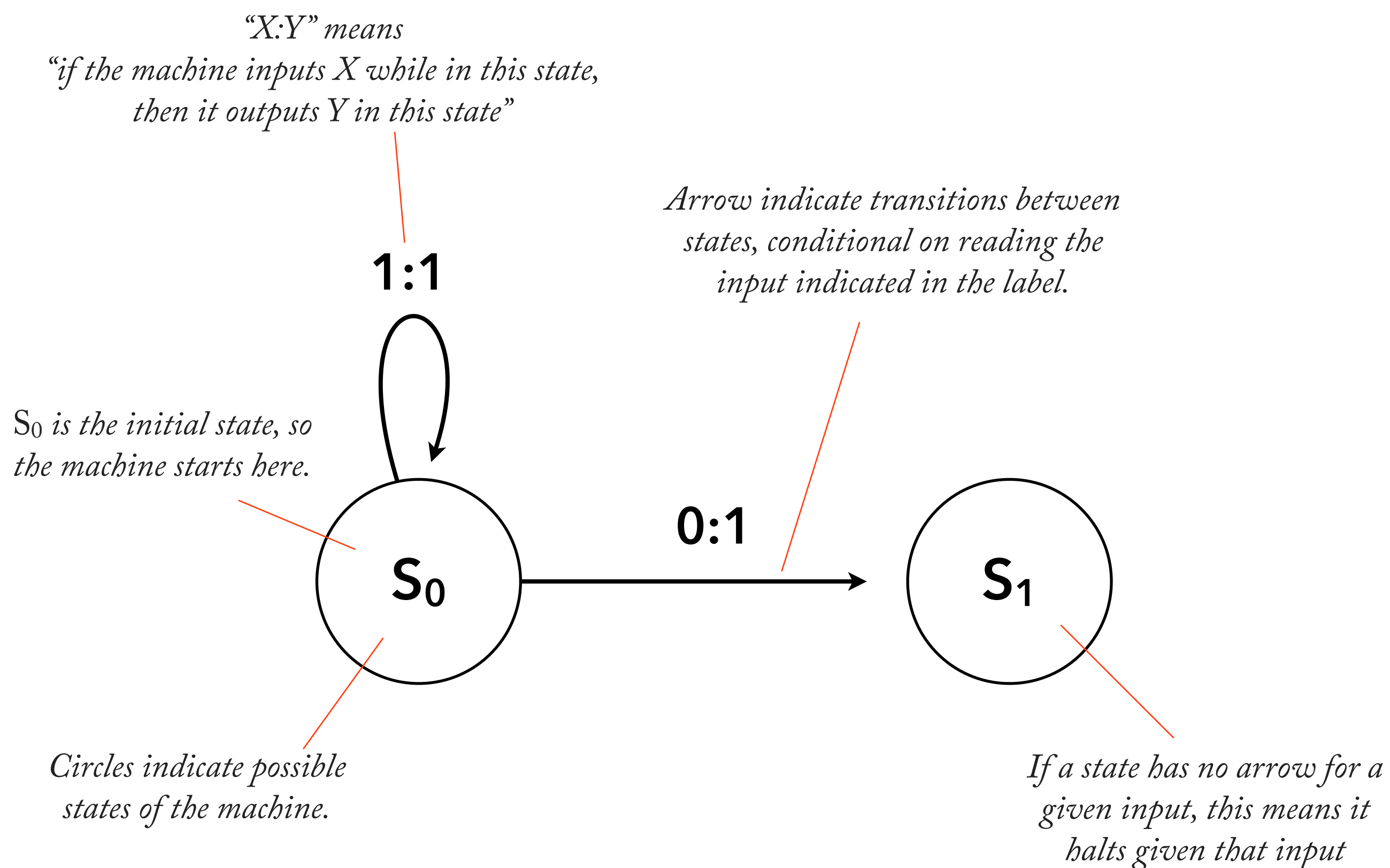


## 24. Finite State Machines

### Reading FSM Diagrams

You can get a feel for how FSM diagrams work by studying the following simple example:



This diagram represents the following rule:

In state  $S_0$ ,  
if input = 1, then output = 1, and return to  $S_0$ ;  
if input = 0, then output = 1, and move to  $S_1$ .

In state  $S_1$ , halt on any input.

## Constructing FSM Diagrams

---

FSM diagrams can be constructed from the following elements.

---

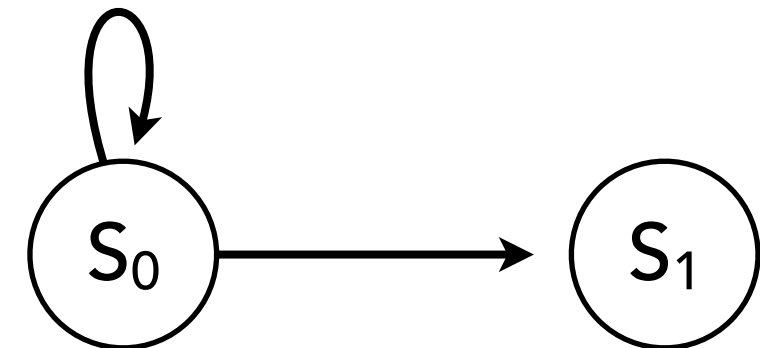
**Nodes:** circles associated with names.

- Each node represents a state.
- The initial state is always named “ $S_0$ ”.



**Links:** one-directional arrows connecting circles to circles.

- There are at most two lines leaving each circle.\*
- There may be zero lines leaving a circle.
- Circles can be connected to themselves.
- The lines can cross.

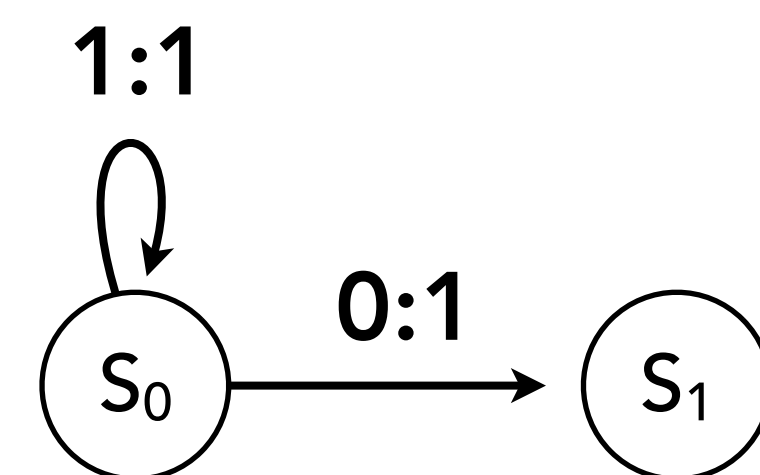


\*More precisely, there are as many lines leaving each circle as possible inputs. If the machine can read  $n$  symbols, there will be up to  $n$  links for each node.

---

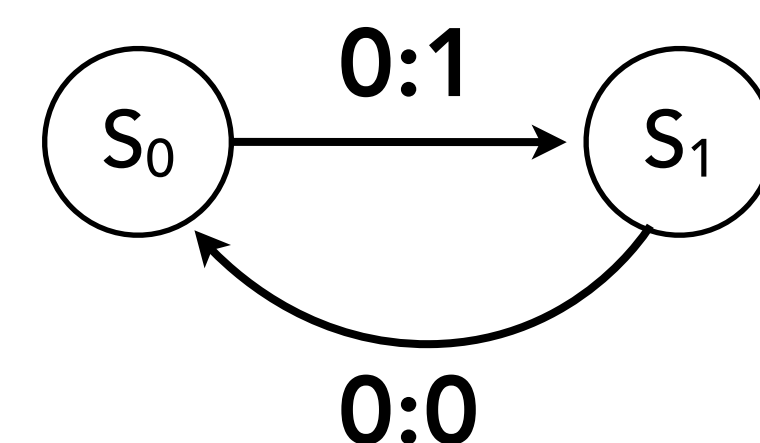
**Labels:** each line is labeled with with an expression of the form “*input : output*”.

- Possible inputs: 1, 0
- Possible outputs: 1, 0.
- **Note!** For each circle, there can be at most one arrows labeled with input 1, and one arrows labeled with input 0.
- A common variation writes the output where we’ve put the state name, and labels the line only with the input.



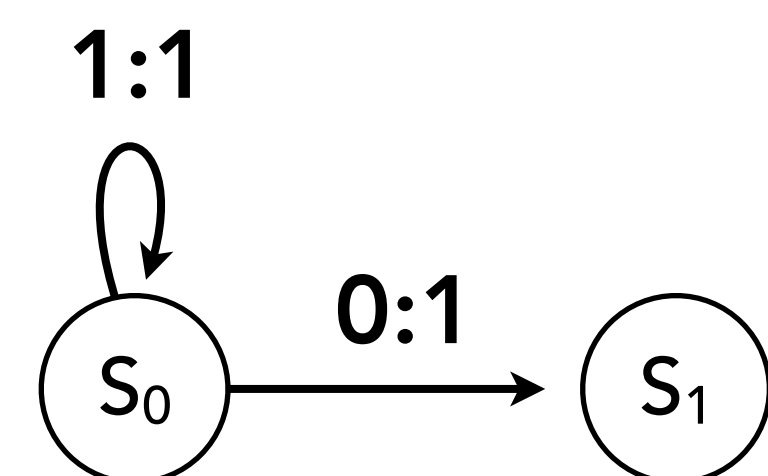
**Loops:** any kind of loop is acceptable:

- Loops from a state to itself.
- Loops among a groups of states.



**Halting:**

- A machine halts in a given state, given an input, if there are no arrows leaving that state for that input.
- If there are *no* lines leaving a state, the machine will halt in that state no matter what.



## Computing with Finite State Machines

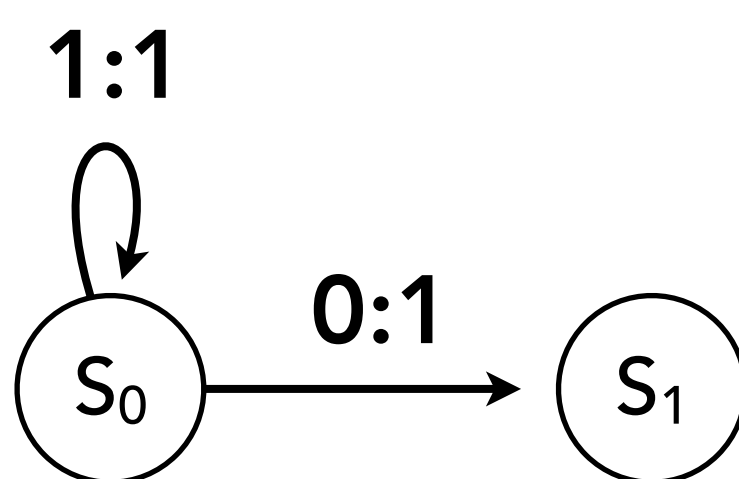
---

Computation for FSMs is defined in the same way as it is for SCs. Local inputs and outputs are processed one bit at a time, and the global input and output corresponds to the sequences of inputs and outputs which is reached when all further input and output bits will be 0s.

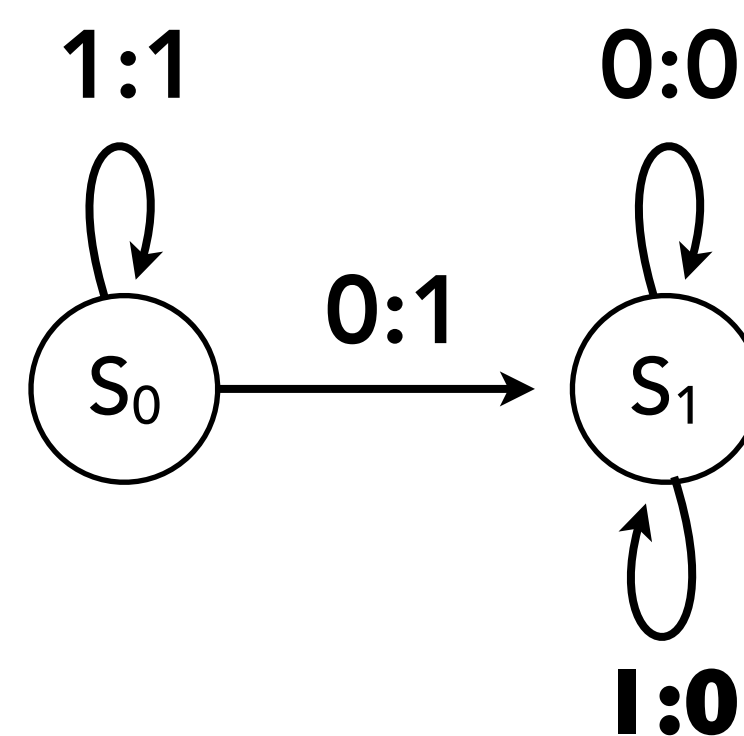
Note on halting:

- A machine **halts** in a given state, given an input, if there are no arrows leaving that state for that input.
- When computing a function, an FSM should never halt. Instead, it should end computation by outputting 0's indefinitely.

So **don't** design your FSM this way.

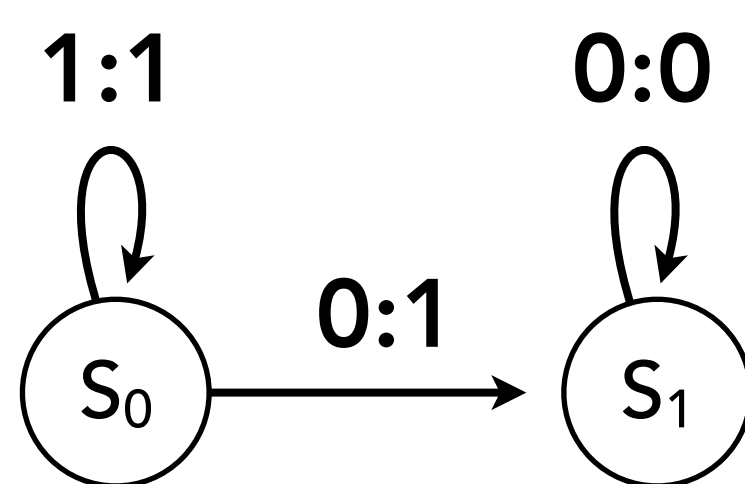


**Do** ensure that the final state produces a continuous output of 0's:



---

To compute tally successor...

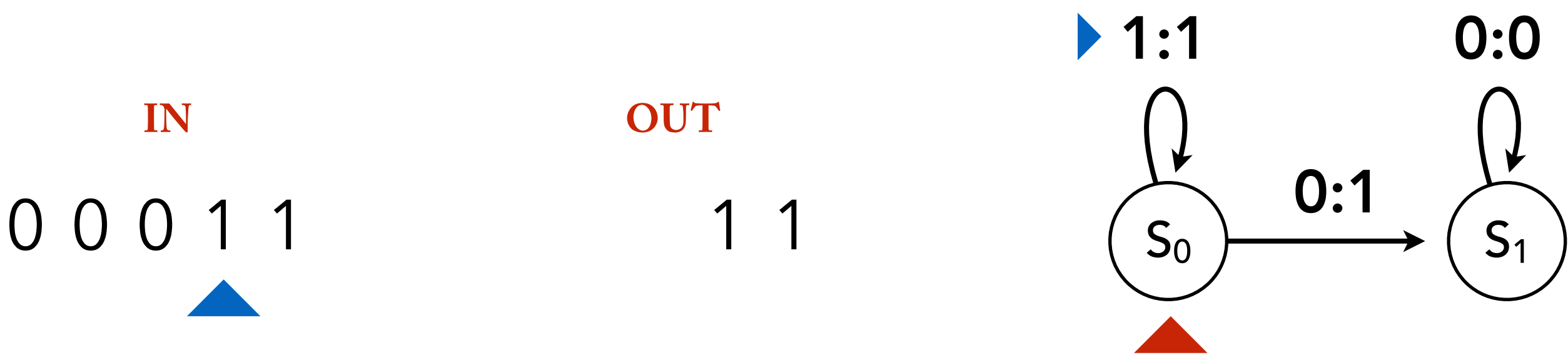
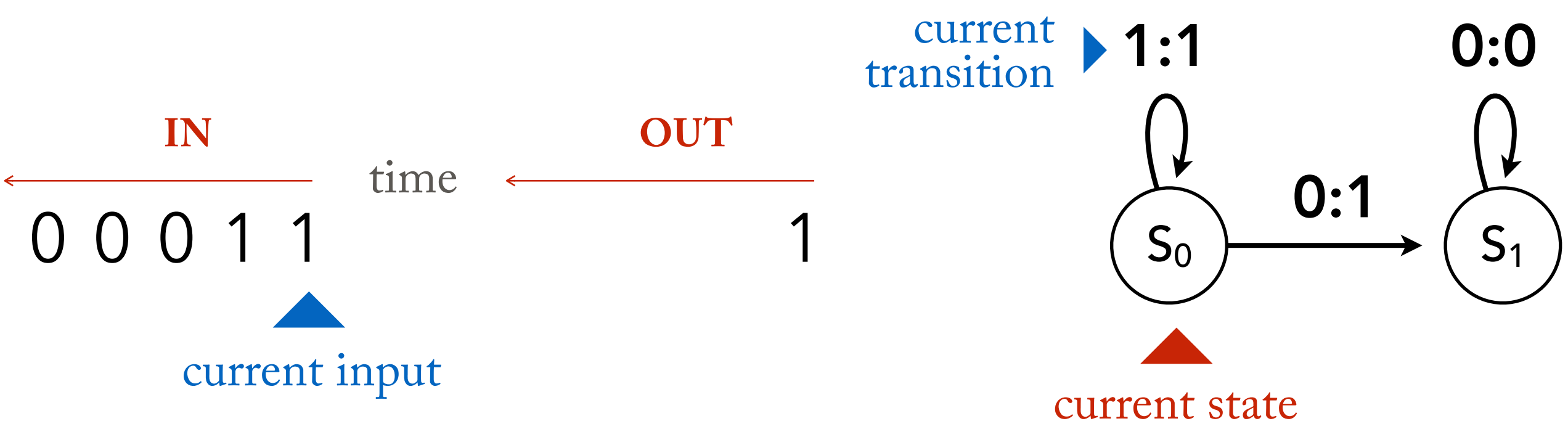
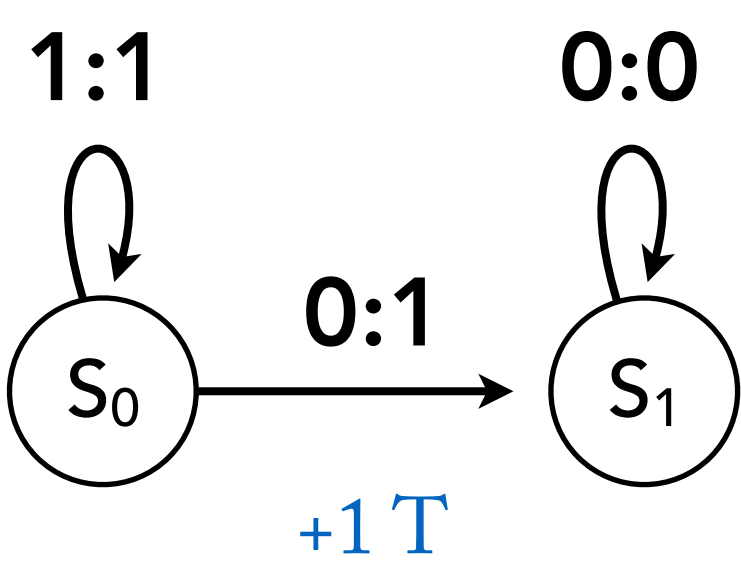


+1 T

Example: + I T

This FSM at right computes successor in tally.

The timeline below shows the step-by-step progression of the machine for a sample input.



Example: +1 B

This FSM at right computes successor in binary.

The timeline below shows the step-by-step progression of the machine for a sample input.

